



In the name of God

University of Tehran
Faculty of Electrical and Computer Engineering

Deep Generative Models

Homework 2

Name and Surname	Mohammad Taha Majlesi Kopaei
Student ID	810101504
Submission Date	2024/12/28

Contents

1	Question 1 - Flow-Based Models	2
1.1	Part 1	2
1.1.1	Part 1 Code	2
1.1.2	Subsection 2	4
1.2	Subsection 3	6
1.2.1	Out-of-Distribution Detection	6
1.3	Subsection 4	8
1.4	Subsection 5 (Bonus)	10
2	Part 2	13
2.1	Answer to the first subsection:	13
2.1.1	Residual Flows	13
2.2	Answer to the second subsection:	14
2.2.1	Calculating the Jacobian Determinant in Residual Flows	14
3	Question 2	16
3.1	Subsection 1	16
3.2	Subsection 2	17
3.3	Subsection 3	17
4	Part 2 - Wasserstein GAN	19
4.1	Subsection 1	19
4.2	Subsection 2	19
4.3	Subsection 3: Evaluation Metrics	20
4.3.1	1. Kullback-Leibler Divergence (KL Divergence)	20
4.3.2	2. Jensen-Shannon Divergence (JS Divergence)	20
4.3.3	3. Wasserstein Distance	20
4.4	Subsection 4	20
4.4.1	1. Loss Value of the Critic	20
4.4.2	2. Sample Density	21
4.4.3	3. Visual Evaluation	21
4.4.4	4. Using Additional Metrics like FID (Fréchet Inception Distance)	21
4.4.5	5. Monitoring Convergence in Critic and Generator	21
5	Question 2 - Implementation of GAN Models	22
5.1	Subsection 1	22
5.1.1	GAN Network Implementation on Fashion MNIST Data	22
5.2	Subsection 2	23
5.2.1	Explaining the ConvTranspose2d Layer in the Generator	23
5.3	Subsection 3	24
5.3.1	2. Generating and Displaying Samples	26
5.4	Subsection 4: Plotting Generator and Discriminator Loss	27
5.5	Subsection 5: FID Explanations	27
5.5.1	FID (Fréchet Inception Distance) Metric	27
5.6	Subsection 6	29

1 Question 1 - Flow-Based Models

1.1 Part 1

1.1.1 Part 1 Code

```
1 class CouplingLayer(nn.Module):
2     def __init__(self, input_dim, hidden_dim, mask):
3         super(CouplingLayer, self).__init__()
4         self.register_buffer('mask', mask)
5         self.scale_net = nn.Sequential(
6             nn.Linear(input_dim, hidden_dim),
7             nn.ReLU(),
8             nn.Linear(hidden_dim, input_dim),
9             nn.Tanh()
10        )
11        self.translate_net = nn.Sequential(
12            nn.Linear(input_dim, hidden_dim),
13            nn.ReLU(),
14            nn.Linear(hidden_dim, input_dim)
15        )
16
17    def forward(self, x):
18        x_masked = x * self.mask
19        s = self.scale_net(x_masked) * (1 - self.mask)
20        t = self.translate_net(x_masked) * (1 - self.mask)
21        y = x_masked + (1 - self.mask) * (x * torch.exp(s) + t)
22        log_det_jacobian = (s * (1 - self.mask)).sum(dim=1)
23        return y, log_det_jacobian
24
25    def inverse(self, y):
26        y_masked = y * self.mask
27        s = self.scale_net(y_masked) * (1 - self.mask)
28        t = self.translate_net(y_masked) * (1 - self.mask)
29        x = y_masked + (1 - self.mask) * (y - t) * torch.exp(-s)
30        return x
```

This code implements a `CouplingLayer`, which is a fundamental component in Normalizing Flows networks. The purpose of this layer is to transform the distribution of input data into a simpler distribution (like a standard normal distribution) in an invertible manner. This feature is very useful for modeling the probability density of data and generating new data.

Functionality:

- **Mask:** A portion of the data is kept constant, and transformations are applied only to a specific part.
- **Scaling and Translation Transformations:** The variable data is modified by:
 1. **Scaling** to change their values.
 2. **Translation** to shift the values.
- **Invertibility:** This layer is designed in such a way that the original data can be reconstructed from the transformed data.

Applications:

- **Probability Density Modeling:** Learning the exact distribution of data.
- **New Data Generation:** Creating data similar to real data.
- **Data Analysis:** Finding hidden structures in the data distribution.

```
1 class RealNVP(nn.Module):
2     def __init__(self, input_dim, hidden_dim, num_coupling_layers):
3         super(RealNVP, self).__init__()
4         self.input_dim = input_dim
5         self.hidden_dim = hidden_dim
```

```

6     self.num_coupling_layers = num_coupling_layers
7
8     masks = []
9     for i in range(num_coupling_layers):
10         mask = np.zeros(input_dim)
11         if i % 2 == 0:
12             mask[::2] = 1
13         else:
14             mask[1::2] = 1
15         mask = torch.from_numpy(mask.astype(np.float32))
16         masks.append(mask)
17
18     self.layers = nn.ModuleList()
19     for i in range(num_coupling_layers):
20         self.layers.append(CouplingLayer(input_dim, hidden_dim, masks[i]))
21
22     def forward(self, x):
23         log_det_jacobian = 0
24         for layer in self.layers:
25             x, ldj = layer(x)
26             log_det_jacobian += ldj
27         return x, log_det_jacobian
28
29     def inverse(self, z):
30         for layer in reversed(self.layers):
31             z = layer.inverse(z)
32         return z

```

This code implements the **RealNVP** model, which is considered one of the models for Normalizing Flows. The goal of this model is to transform a complex distribution into a simpler distribution (like a normal distribution) in an invertible way. This model uses **Coupling Layers**, which were explained in the previous code.

Functionality:

1. Inputs:

- **input_dim**: Number of input features.
- **hidden_dim**: Number of neurons in the hidden layer.
- **num_coupling_layers**: Number of coupling layers.

2. **Masks**: Masks are created alternately (even and odd features are kept constant or changed alternately). These masks help to transform the data in a controlled manner.

3. **Model Structure**: The model consists of several **Coupling Layers**, each of which transforms a part of the input and preserves invertibility. The layers are stored using **nn.ModuleList** to be applied sequentially on the data.

4. **Forward Process**: The input **x** is passed through all the coupling layers. In each layer, the data is transformed, and the determinant of the Jacobian (for probability density) is calculated. The final output includes the transformed data **y** and the sum of the determinants of the Jacobians.

5. **Inverse Process**: The output data **z** is passed through the same layers but in reverse order, returning to the original input **x**.



Figure 1: Model Parameters

Parameter Definitions for the Model: As seen, the number of coupling layers is defined as 8.

1.1.2 Subsection 2

```

1 train_dataset = torchvision.datasets.FashionMNIST(root='./data', train=True, download=
  True, transform=transform)
2 batch_size = 128
3
4 train_size = int(0.8 * len(train_dataset))
5 validation_size = len(train_dataset) - train_size
6 train_subset, validation_subset = random_split(train_dataset, [train_size,
  validation_size])
7
8 train_loader = DataLoader(train_subset, batch_size=batch_size, shuffle=True)
9 test_loader = DataLoader(validation_subset, batch_size=batch_size, shuffle=False)

```

In this part, the S and T sections are defined.

```

1 class CouplingLayer(nn.Module):
2     def __init__(self, input_dim, hidden_dim, mask):
3         super(CouplingLayer, self).__init__()
4         self.register_buffer('mask', mask)
5         self.scale_net = nn.Sequential(
6             nn.Linear(input_dim, hidden_dim),
7             nn.ReLU(),
8             nn.Linear(hidden_dim, input_dim),
9             nn.Tanh()
10        )
11        self.translate_net = nn.Sequential(
12            nn.Linear(input_dim, hidden_dim),
13            nn.ReLU(),
14            nn.Linear(hidden_dim, input_dim)
15        )

```

Architectural Details: Both networks consist of three main layers:

1. First Layer (Linear Layer):

- The number of input neurons is equal to `input_dim`, i.e., the number of input features.

- The number of output neurons is equal to `hidden_dim`, which provides a higher-dimensional representation space for learning features.
2. **Non-linear Activation Function (ReLU):**
- To increase the model's learning capacity, a ReLU (Rectified Linear Unit) is used, which adds non-linear features to the network.
3. **Second Layer (Linear Layer):**
- The number of input neurons is equal to `hidden_dim`, and the number of outputs is equal to `input_dim`, meaning it returns to the original data dimensions.
4. **Output Function:**
- For the S network, the output is limited by `tanh`. This function causes the scaling values to be in the range $(-1, 1)$, and the model's transformations remain stable.
 - For the T network, the output is without limitation, as the translation amount can be in any range.

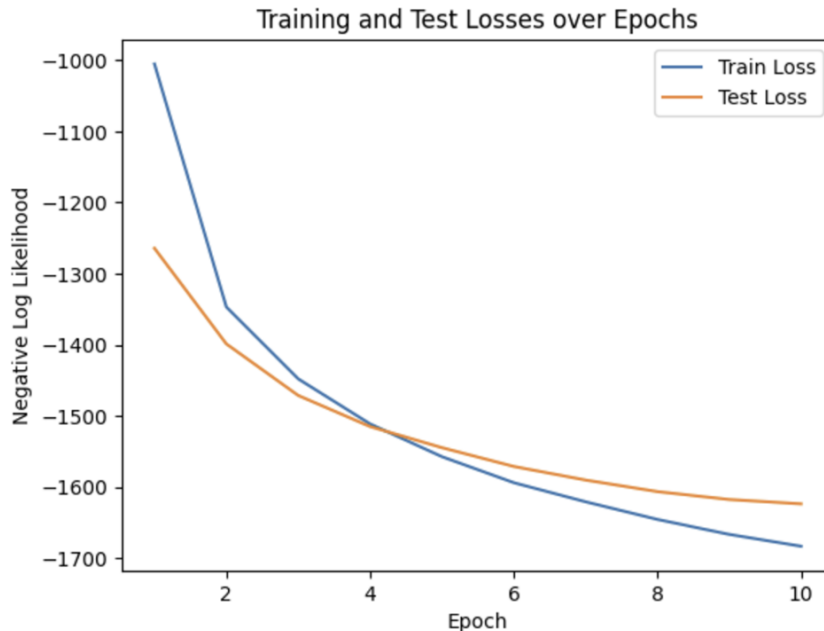


Figure 2: Training and Test Losses over Epochs

Chart Analysis:

1. **Training Error (Train Loss - Blue Line):**

- This line shows how the model learns the training data.
- The training error value is initially high but decreases over time and with an increasing number of epochs, which indicates better learning by the model.

2. **Test Error (Test Loss - Orange Line):**

- This line shows the model's error on the test data (data that the model has not seen during training).
- The test error also decreases similarly to the training error, which shows that the model performs well in learning new data and generalizing to unseen data.

Error Reduction:

- Both training and test errors decrease during the epochs, which shows that the model is learning correctly.
- The decrease in test error along with training means that the model has not overfitted and generalizes well to new data.

Convergence:

- The error reduction is faster in the initial epochs (up to about epoch 5), but after that, the rate of error reduction decreases. This behavior indicates that the model is approaching convergence.

Good Performance:

- The test error is close to the training error, which indicates that the model predicts unseen data well.

Some of the images generated by the model at the end will be as follows:



Figure 3: Generated Images

1.2 Subsection 3

1.2.1 Out-of-Distribution Detection

Out-of-distribution detection is one of the key challenges in machine learning systems. This topic means identifying data that is clearly different from the distribution of data on which the model was trained. This data can cause the model to perform poorly, especially in fields such as medicine, autonomous cars, and cybersecurity, where a wrong decision can have serious consequences.

Concept of Out-of-Distribution Data

1. **In-Distribution Data:** Data that comes from the same statistical distribution that the model was trained on. This data should be processed well by the model.
2. **Out-of-Distribution Data:** Data that comes from a completely different statistical distribution than the training data. The model usually does not have a good ability to predict or process this data.

Goal: To design a model that can detect out-of-distribution data and prevent wrong decision-making.

Probability-Based Methods for Out-of-Distribution Detection One of the main approaches in detecting out-of-distribution data is the use of probabilistic models. These models assign a likelihood to the data, and data that have a very low probability are identified as out-of-distribution data.

Examples of Probabilistic Models:

1. **Normalizing Flows:** Models like RealNVP and Glow transform data into a simple distribution (usually Gaussian) and calculate the probability of each data point. Data whose distribution is different from the training data receive a lower probability and are identified as out-of-distribution data.
2. **Autoregressive Models:** Models like PixelCNN or WaveNet model the conditional probability of each part of the data (like each pixel). These models assign a lower probability to unfamiliar data.
3. **Energy-Based Models:** These models assign an energy value to the data. In-distribution data have lower energy, while out-of-distribution data will have higher energy. Using energy score instead of raw probability helps to improve the detection of out-of-distribution data.
4. **Probabilistic Deep Learning Models:** Low-variance models like Bayesian Neural Networks use model uncertainty to detect out-of-distribution data. Data on which the model has low certainty can be identified as out-of-distribution data.

Existing Methods: Energy-Based Models Energy models are one of the effective methods for detecting out-of-distribution data. In these models, an energy value is assigned to each data point, which indicates the level of "fit" of that data with the model.

- **Main Idea:** In-distribution data usually have low energy because the model has learned them well. Out-of-distribution data have higher energy because the model cannot represent them well.

Reasons for the Failure of Some Models in Detecting Out-of-Distribution Data

1. **Assigning High Probability to Out-of-Distribution Data:** Probability-based models like Glow and RealNVP sometimes assign high probabilities to out-of-distribution data. For example, they might assign a high probability to simple images or white noise because their structure is simpler.
2. **Problem of the Raw Probability Metric:** Raw probability cannot effectively distinguish in-distribution data from out-of-distribution data. For this reason, using complementary metrics (like energy score or a combination of probabilities) is necessary.
3. **Dependence on the Training Data Distribution:** Models that are trained solely based on in-distribution data do not have the ability to generalize to new and different data.
4. **Lack of Learning Sufficient Features:** If the training data has little diversity, the model cannot learn the key features of the in-distribution data well, which makes detecting out-of-distribution data more difficult.
5. **Lack of Direct Optimization for Out-of-Distribution Data:** Many models are designed only for learning in-distribution data and have no direct mechanism for identifying unfamiliar data.

1.3 Subsection 4

```
1 def compute_log_likelihood(model, data_loader):
2     model.eval()
3     log_likelihoods = []
4     with torch.no_grad():
5         for data, _ in data_loader:
6             data = data.view(-1, input_dim).to(device)
7             z, log_det_jacobian = model(data)
8             log_pz = -0.5 * torch.sum(z**2 + np.log(2 * np.pi), dim=1)
9             log_px = log_pz + log_det_jacobian
10            log_likelihoods.append(log_px.cpu().numpy())
11    return np.concatenate(log_likelihoods)
```

This function calculates the log-likelihood of the input data, which is a measure of the probability of this data relative to the distribution learned by the model. Its functionality can be divided into several stages:

1. Transforming Data to Latent Space:

- The model maps the input data to a simpler latent space (usually a Gaussian distribution with zero mean and unit variance).
- This transformation allows the model to represent complex data in a simpler form.

2. Calculating the Probability of Data in the Latent Space ($\log p(z)$):

- The probability of the data in the latent space is calculated based on the assumption of a Gaussian distribution.
- Data values that match this distribution well will have a higher probability.

3. Returning to the Original Space ($\log p(x)$):

- To return to the original data space, the effect of the changes caused by the model is considered through the determinant of the Jacobian.
- This value represents the effect of transforming the data from the original space to the latent space and is added to the probability of the data in the original space.

4. Final Calculation:

- The combination of the probability of the latent space ($\log p(z)$) and the effect of the transformation ($|\log |\det J||$) gives the log-likelihood in the original space ($\log p(x)$).

Application:

- **Out-of-Distribution (OOD) Detection:** Data that have a lower probability (lower log-likelihood) are usually outside the distribution of the training data.
- **Model Evaluation:** It checks how the model explains the input data and whether it has an appropriate generalization ability.

Fashion MNIST Train Log Likelihood: 1706.9332
Fashion MNIST Test Log Likelihood: 1621.3966
MNIST Log Likelihood: 890.0156
KMNIST Log Likelihood: -1850.9489

As can be seen, for the data from the train set, the average likelihood value is higher than the test data. But for the MNIST data, we expected the average value to be lower because the number data have no connection or similarity to the Fashion MNIST data, and we expected the model to assign lower probabilities to them. And in the KMNIST data, the average value has decreased, which indicates that lower probabilities are given to this data. So in general we have:

Analysis of Log-Likelihood Values for Different Data In this experiment, the average log-likelihood values for the FashionMNIST (training and test), MNIST, and KMNIST datasets were calculated and compared. These values indicate the amount of probability assigned by the model to this data. The analysis of the results is as follows:

1. FashionMNIST Data (Training and Test):

- The log-likelihood value for the FashionMNIST training data is higher than the test data.
- **Analysis:** This behavior is natural because the model has been trained on the training data and has learned them well, so it assigns a higher probability to this data. The test data are from the same distribution, but the model has never seen them, for this reason, they receive a lower probability than the training data. This difference indicates the proper generalization of the model.

2. MNIST Data:

- The average log-likelihood for the MNIST data is 890.0156.
- **Analysis:** It was expected that this value would be much lower because the MNIST data in terms of content (numbers) are very different from the FashionMNIST data (images of clothes). This behavior indicates one of the problems of generative models like RealNVP. These models sometimes assign a high probability to data that are simpler (like the numbers in MNIST), even if this data is out of the main distribution. The reason for this issue could be the simple structure of the MNIST data and its statistical similarities (like brightness or simple patterns).

3. KMNIST Data:

- The average log-likelihood for the KMNIST data is -1850.9489.
- **Analysis:** The model has assigned a very low probability to the KMNIST data, which includes Japanese characters. This behavior is expected because the KMNIST data, both in terms of content (Japanese characters) and structure (more complex patterns), are completely different from FashionMNIST. The very low log-likelihood value indicates that the model has identified this data as out-of-distribution.

Overall Conclusion:

1. The model assigns a higher probability to the training data, which indicates successful learning of the in-distribution data.
2. The log-likelihood value is lower for the test data but still acceptable, which indicates the model's proper performance in generalizing to unseen data.
3. Assigning a relatively high probability to the MNIST data indicates one of the model's limitations. This model assigns a high probability to simple and statistically similar data (like the numbers in MNIST), even if they are not from the main distribution.
4. Assigning a very low probability to the KMNIST data shows that the model can identify data that are completely out of the main distribution. This behavior indicates the model's ability to detect irrelevant data.

Suggestion for Improvement:

- Use of energy methods (Energy-Based Models) or a combination of several criteria to reduce the problem of assigning high probability to simple data like MNIST.
- Training the model on more diverse data or using regularization techniques to improve performance in detecting out-of-distribution data.

1.4 Subsection 5 (Bonus)

```
1 class Decoder(nn.Module):
2     def __init__(self, latent_dim, output_dim):
3         super(Decoder, self).__init__()
4         self.decoder = nn.Sequential(
5             nn.Linear(latent_dim, 512),
6             nn.ReLU(),
7             nn.Linear(512, output_dim)
8         )
9     def forward(self, x):
10        return self.decoder(x)
11
12 class Encoder(nn.Module):
13     def __init__(self, input_dim, latent_dim):
14         super(Encoder, self).__init__()
15         self.encoder = nn.Sequential(
16             nn.Linear(input_dim, 512),
17             nn.ReLU(),
18             nn.Linear(512, latent_dim)
19         )
20     def forward(self, x):
21        return self.encoder(x)
```

Initially, we trained an encoder and a decoder so that they could take the images to their Latent section. That is, the images that are taken will be taken to the latent space by the encoder.

```
1 encoder = Encoder(input_dim, latent_dim).to(device)
2 decoder = Decoder(latent_dim, input_dim).to(device)
3 encoder_decoder_optimizer = optim.Adam(list(encoder.parameters()) + list(decoder.
4     parameters()), lr=learning_rate)
5
6 for epoch in range(num_epochs_encoder):
7     # ... training loop ...
```

In the next step, we trained it with the train data so that we could bring that data to a roughly specific part of the latent space.

```
1 def encode_dataset(encoder, data_loader):
2     # ... encoding logic ...
3
4 # Encode datasets
5 fashion_train_latent = encode_dataset(encoder, train_loader)
6 # ... and so on for other datasets
```

Now we have defined a function that encodes the data to the latent space with the encoder:

```
1 latent_input_dim = latent_dim
2 realnvp_model = RealNVP(latent_input_dim, hidden_dim, num_coupling_layers).to(device)
3 realnvp_optimizer = optim.Adam(realnvp_model.parameters(), lr=learning_rate)
4
5 for epoch in range(num_epochs):
6     # ... training loop for RealNVP on latent data ...
```

In this part, we trained the `realNVP` model with the data from the LATENT space, so that the model learns to map the data from the latent space to the original images. This means it's not like the previous case where we went from the standard normal space to the image space; we are mapping from the Latent space of an encoder that was previously trained on the images to the main data. So it should be logical that other data that were not learned by the encoder would go to a sparse space in the latent space, and in the same proportion, when giving the probability by the flow model, the probability that is assigned to them would be less.

```
1 def compute_log_likelihood_latent(model, data):
2     # ... log-likelihood calculation for latent data ...
3
4 fashion_train_ll = compute_log_likelihood_latent(realnvp_model, fashion_train_latent)
5 # ... and so on for other datasets
```

FashionMNIST Train Log-Likelihood: 166.30286
FashionMNIST Test Log-Likelihood: 151.63997
MNIST Log-Likelihood: 2.3912132
KMNIST Log-Likelihood: -66.17415

As we can see, according to the results, we have reached the correct answer, meaning that less value was assigned to the data outside the main distribution.

Analysis of Log-Likelihood Results The calculated log-likelihood results for different datasets are as follows:

- **FashionMNIST Train Log-Likelihood: 166.30286**
- **FashionMNIST Test Log-Likelihood: 151.63997**
- **MNIST Log-Likelihood: 2.3912132**
- **KMNIST Log-Likelihood: -66.17415**

Now we analyze these values:

1. FashionMNIST (In-Distribution):

- The log-likelihood for the training data (166.30) is the highest, which shows that the model has learned the training data well.
- The log-likelihood for the test data (151.63) is lower than the training data but still high. This is a completely natural difference between training and test data and shows that the model has good generalization ability. The high log-likelihood values for both sets indicate that the model correctly describes the in-distribution data.

2. MNIST (Out-of-Distribution):

- The log-likelihood for MNIST (2.39) is much lower than for FashionMNIST. This is a very significant improvement compared to the previous state.
- The low log-likelihood for MNIST shows that the model was able to identify this data as out-of-distribution (OOD). This indicates that using the latent space has reduced the superficial similarities (like brightness and simple patterns) between the MNIST and FashionMNIST data.

3. KMNIST (More Different OOD):

- The log-likelihood for KMNIST (-66.17) is even lower than MNIST. KMNIST contains Japanese characters, which are structurally very different from FashionMNIST.
- The very low (negative) log-likelihood for KMNIST shows that the model has identified this data as completely out-of-distribution.

Effect of Using the Latent Space

- Transferring the data to the latent space with an encoder has caused the model to better understand the meaningful features and detect out-of-distribution data more accurately.
- Compared to the direct implementation of RealNVP on raw data:
 1. The log-likelihood value for the FashionMNIST data is still high, which shows that the model has preserved its accuracy for in-distribution data.
 2. A significant difference is observed between the log-likelihood of the MNIST and KMNIST data, which indicates the model's better ability to distinguish out-of-distribution data.

Suggestions for Improvement

- If the distance between MNIST and FashionMNIST in the log-likelihood is less than expected, the encoder can be improved with more precise settings (like adding more layers or using Reconstruction Loss).
- The model can be strengthened for more accurate OOD detection by combining RealNVP with energy-based methods (Energy-Based Models).

Final Summary

- The model, using the latent space, was able to explain the in-distribution data well and accurately identify the out-of-distribution data (especially KMNIST).
- These results show that the use of an encoder and latent space was successful and has improved the model's performance in detecting out-of-distribution data.

2 Part 2

2.1 Answer to the first subsection:

2.1.1 Residual Flows

Residual flow models are one of the advanced learning models designed for modeling complex distributions and generating precise samples from data. The main goal of these models is to use invertible transformations that, while preserving accuracy, simplify complex calculations. Let's explain the structure of these models step by step.

What does invertibility mean? Suppose we want to transform data from a simple space (like a normal distribution) to a complex space (like an image) or vice versa. For this, we need the transformation to be such that:

1. It is one-to-one (i.e., each input has a unique output and vice versa).
2. It is reversible (we can get the input back from the output).

Residual flow models use this property of reversibility to model complex data distributions with transformations that are easily calculable.

Model Structure The main structure of these models is based on an invertible residual block, which is defined as follows:

$$y = f(x) = x + g(x)$$

- x : input to the block.
- y : output of the block.
- $g(x)$: a simple function (usually a neural network) that applies the necessary changes to x .

The important feature of this formula is that since $g(x)$ is a simple function, we can easily reverse $f(x)$ (invert it).

How is invertibility guaranteed? For $f(x)$ to be invertible, we need $g(x)$ to be a contraction mapping. This means that when different inputs are applied, these inputs do not get too close to each other. In simple terms, if the distance between two points x_1 and x_2 is d , after applying $g(x)$, the distance between the outputs should be less than d .

$$||g(x_1) - g(x_2)|| \leq L||x_1 - x_2||$$

where L must be less than 1.

How do we find the inverse? To find the inverse of $f(x)$, it is enough to solve the following equation:

$$x = f^{-1}(y) = y - g(x)$$

Since $g(x)$ depends on x , this equation is not directly solvable, but we can use a simple method called fixed-point iteration:

1. We choose an initial value for x (usually $x_0 = y$).
2. We update the value of x with the following relation: $x_{n+1} = y - g(x_n)$.
3. We repeat this until the value of x converges to the exact answer.

How do we make $g(x)$ a contraction? To ensure that $g(x)$ is a contraction, we use spectral normalization. This method adjusts the weight matrices in $g(x)$ so that their largest singular value is less than 1. This is done in three steps:

1. We calculate the largest singular value (σ).
2. We normalize the weights: $W_{normalized} = \frac{W}{\sigma(W)}$.
3. We multiply the normalized value by a coefficient $c < 1$.

Advantages of these models:

- **Exact Invertibility:** Due to the contraction property of $g(x)$, we can reverse any transformation easily.
- **Computational Efficiency:** There is no need to store large matrices or perform complex calculations.
- **Accurate Modeling:** These models can learn complex data distributions with high accuracy.

2.2 Answer to the second subsection:

2.2.1 Calculating the Jacobian Determinant in Residual Flows

One of the key issues in flow models is the calculation of the Jacobian determinant, which indicates the changes in probability density when a transformation is applied. In residual flow models, instead of heavy and direct calculations, intelligent methods like the Hutchinson's estimator and Russian roulette are used. In the following, we explain these concepts simply and fluently.

Why do we need the Jacobian determinant? When a transformation $f(x)$ is applied to the data, their probability density changes. Based on the change of variables theorem, the new density is calculated as follows:

$$f(x) = x + g(x)$$

Here:

- $Jf(x)$ is the Jacobian of the transformation $f(x)$.
- $\log \det(Jf(x))$ shows the change in probability density resulting from the transformation.

Therefore, for correct modeling, we need to calculate $\log \det(Jf(x))$.

Structure of the Transformation in a Residual Flow In residual flow models, the transformation is defined as follows:

$$f(x) = x + g(x)$$

- x : input.
- $g(x)$: a simple function (usually a neural network) that applies the changes.

The Jacobian of this transformation is equal to:

$$Jf(x) = \frac{\partial f}{\partial x} = I + Jg(x)$$

where I is the identity matrix and $Jg(x)$ is the Jacobian of the function $g(x)$.

How do we calculate $\log \det(Jf(x))$? Calculating the determinant of $Jf(x)$ directly, especially for high-dimensional data, is very costly. In residual flow models, this problem is solved by rewriting the determinant as a power series:

$$\log \det(Jf(x)) = \text{Tr}(\log(I + Jg(x))) = \sum_{k=1}^{\infty} \frac{(-1)^{k+1}}{k} \text{Tr}([Jg(x)]^k)$$

Here, Tr denotes the trace of the matrix. This power series allows us to estimate the determinant without calculating it directly.

Using the Hutchinson's Estimator Calculating the trace of $\text{Tr}([Jg(x)]^k)$ can also be costly. Therefore, we use an estimation method called the Hutchinson's estimator. The idea of this estimator is that instead of calculating the trace directly, we calculate it using an inner product:

$$\text{Tr}(A) = E_v[v^T A v]$$

- v : a random vector with standard normal elements (zero mean and unit variance).
- $v^T A v$: an inner product that provides an estimate of the trace of the matrix A .

Therefore, each term in the power series is estimated as follows:

$$\text{Tr}([Jg(x)]^k) \approx E_v[v^T [Jg(x)]^k v]$$

Reducing the Number of Series Terms with "Russian Roulette" Calculating an infinite number of terms in the series is impossible. To solve this problem, a technique called Russian roulette is used. This method randomly determines at which term the series calculation should stop, while ensuring that the final estimate is unbiased. Russian roulette works as follows:

1. **Choosing a random number:** The number of terms to be calculated in the series is determined based on a random distribution (like a geometric distribution).
2. **Weighting the terms:** Each term is scaled based on its probability of being present in the series.

Advantages of this method:

1. **Reduced computational cost:** There is no need to calculate the determinant directly or a large number of series terms.
2. **High accuracy:** The final estimate is unbiased.
3. **Memory savings:** Instead of storing large matrices, an inner product with random vectors is used.

Summary In residual flow models, the Jacobian determinant is calculated efficiently using a power series and the Hutchinson's estimator. The use of the Russian roulette method also reduces the computational cost, while preserving the accuracy of the estimate. This approach not only simplifies the calculations but also makes it possible to use more complex models.

3 Question 2

3.1 Subsection 1

In GAN models, the generator tries to create samples that cannot be distinguished from real samples by the discriminator. The generator's loss function is defined as follows:

$$L_G(\theta) = E_{z \sim N(0,1)}[\log(1 - D_\phi(G_\theta(z)))]$$

The goal is to show that when the discriminator is fully capable of distinguishing the generated samples (i.e., $D_\phi(G_\theta(z)) \approx 0$), the gradient is approximately zero.

Generator's Loss Function Gradient The gradient of the generator's loss function with respect to its parameters (θ) is defined as follows:

$$\nabla_\theta L_G = E_{z \sim N(0,I)}[\nabla_\theta \log(1 - D_\phi(G_\theta(z)))]$$

To calculate the gradient, we first use the chain rule:

$$\nabla_\theta \log(1 - D_\phi(G_\theta(z))) = \frac{-1}{1 - D_\phi(G_\theta(z))} \nabla_\theta D_\phi(G_\theta(z))$$

And using the gradient of $D_\phi(G_\theta(z))$ with respect to θ :

$$\nabla_\theta D_\phi(G_\theta(z)) = \nabla_x D_\phi(x)|_{x=G_\theta(z)} \cdot \nabla_\theta G_\theta(z)$$

So the overall gradient is written as follows:

$$\nabla_\theta L_G = E_{z \sim N(0,I)} \left[\frac{-\nabla_x D_\phi(G_\theta(z)) \cdot \nabla_\theta G_\theta(z)}{1 - D_\phi(G_\theta(z))} \right]$$

When $D_\phi(G_\theta(z)) \approx 0$: Assume the discriminator can perfectly distinguish the generated samples, meaning:

$$D_\phi(G_\theta(z)) \approx 0$$

In this case:

- The denominator is approximately equal to 1, so its effect is neutralized.
- But the gradient $\nabla_x D_\phi(G_\theta(z))$ also becomes small.

Why? Because $D_\phi(x)$ uses the sigmoid function:

$$D_\phi(x) = \sigma(h_\phi(x)) = \frac{1}{1 + e^{-h_\phi(x)}}$$

When $D_\phi(G_\theta(z)) \approx 0$, then $h_\phi(G_\theta(z)) \ll 0$ (a very negative number). In this condition:

$$\sigma'(h_\phi(x)) = \sigma(h_\phi(x))(1 - \sigma(h_\phi(x)))$$

For $D_\phi(G_\theta(z)) \approx 0$:

$$\sigma'(h_\phi(x)) \approx 0$$

Therefore, $\nabla_x D_\phi(G_\theta(z)) \approx 0$ will hold.

Conclusion: Ultimately, when the discriminator perfectly distinguishes the generated samples, the gradient $\nabla_\theta L_G$ becomes approximately zero.

$$\nabla_\theta L_G \approx 0$$

This means that the generator no longer receives useful information to improve its performance, and its training stops.

3.2 Subsection 2

Discriminator's Loss Function Definition: The discriminator's loss function is defined as follows:

$$L_D(\phi; \theta) = -E_{x \sim P_{data}(x)}[\log D_\phi(x)] - E_{x \sim P_\theta(x)}[\log(1 - D_\phi(x))]$$

Here:

- $P_{data}(x)$: Real data distribution.
- $P_\theta(x)$: Data distribution generated by the generator.

The goal is to show that in the optimal state, the discriminator acts as follows:

$$D^*(x) = \frac{P_{data}(x)}{P_{data}(x) + P_\theta(x)}$$

Optimization of L_D : To optimize L_D , we rewrite the loss function as follows:

$$L_D(D) = - \int P_{data}(x) \log D(x) dx - \int P_\theta(x) \log(1 - D(x)) dx$$

Here, we consider $D(x)$ as the optimization variable. To find the optimal value, we must take the derivative of this expression with respect to $D(x)$ and set it equal to zero.

Taking the Derivative of L_D : We define the function inside the integral as follows:

$$l(x, D(x)) = -P_{data}(x) \log D(x) - P_\theta(x) \log(1 - D(x))$$

Taking the derivative with respect to $D(x)$:

$$\frac{\partial l}{\partial D} = -\frac{P_{data}(x)}{D(x)} + \frac{P_\theta(x)}{1 - D(x)}$$

Setting the derivative equal to zero:

$$-\frac{P_{data}(x)}{D(x)} + \frac{P_\theta(x)}{1 - D(x)} = 0$$

Solving the Equation: By simplifying, we have:

$$\frac{P_{data}(x)}{D(x)} = \frac{P_\theta(x)}{1 - D(x)}$$

We multiply both sides by $D(x)(1 - D(x))$:

$$P_{data}(x)(1 - D(x)) = P_\theta(x)D(x)$$

Rewriting:

$$P_{data}(x) - P_{data}(x)D(x) = P_\theta(x)D(x)$$

Grouping the $D(x)$ terms:

$$P_{data}(x) = D(x)(P_{data}(x) + P_\theta(x))$$

Solving for $D(x)$:

$$D^*(x) = \frac{P_{data}(x)}{P_{data}(x) + P_\theta(x)}$$

3.3 Subsection 3

Problem Statement: If the output of $D_\phi(x)$ is a sigmoid logit:

$$D_\phi(x) = \sigma(h_\phi(x)) = \frac{1}{1 + e^{-h_\phi(x)}}$$

And if $D(x) = D^*(x)$, show that:

$$h_\phi(x) = \log \frac{P_{data}(x)}{P_\theta(x)}$$

Proof: For $D_\phi(x)$:

$$D_\phi(x) = \frac{1}{1 + e^{-h_\phi(x)}}$$

By substituting the value of $D^*(x)$:

$$\frac{1}{1 + e^{-h_\phi(x)}} = \frac{P_{data}(x)}{P_{data}(x) + P_\theta(x)}$$

We multiply both sides by $1 + e^{-h_\phi(x)}$:

$$1 = \frac{P_{data}(x)}{P_{data}(x) + P_\theta(x)}(1 + e^{-h_\phi(x)})$$

Rewriting:

$$1 = \frac{P_{data}(x)}{P_{data}(x) + P_\theta(x)} + \frac{P_{data}(x)}{P_{data}(x) + P_\theta(x)}e^{-h_\phi(x)}$$

We subtract the first term:

$$1 - \frac{P_{data}(x)}{P_{data}(x) + P_\theta(x)} = \frac{P_{data}(x)}{P_{data}(x) + P_\theta(x)}e^{-h_\phi(x)}$$

We simplify the left side:

$$\frac{P_\theta(x)}{P_{data}(x) + P_\theta(x)} = \frac{P_{data}(x)}{P_{data}(x) + P_\theta(x)}e^{-h_\phi(x)}$$

We multiply both sides by $P_{data}(x) + P_\theta(x)$:

$$P_\theta(x) = P_{data}(x)e^{-h_\phi(x)}$$

Solving for $h_\phi(x)$:

$$e^{-h_\phi(x)} = \frac{P_\theta(x)}{P_{data}(x)}$$

We take the logarithm:

$$-h_\phi(x) = \log \frac{P_\theta(x)}{P_{data}(x)}$$

And finally:

$$h_\phi(x) = \log \frac{P_{data}(x)}{P_\theta(x)}$$

Summary:

1. **Subsection 1:** When $D_\phi(G_\theta(z)) \approx 0$, the generator's loss function gradient is approximately zero.
2. **Subsection 2:** The optimal discriminator returns the ratio of the real data probability to the sum of the real and generated data probabilities.
3. **Subsection 3:** The logit output of the discriminator is equivalent to the logarithm of the ratio $\frac{P_{data}(x)}{P_\theta(x)}$.

4 Part 2 - Wasserstein GAN

4.1 Subsection 1

WGAN (Wasserstein GAN) is designed using the Wasserstein distance to solve the problems in standard GANs. This model aims to improve the stability of the training process, solve the problem of vanishing gradients, and provide higher quality generated samples.

Problems in Standard GANs and the WGAN Solution Standard GAN Problems:

1. **Instability in the Training Process:** In standard GANs, training is in the form of a min-max game between the generator and the discriminator. This structure usually leads to severe fluctuations and unpredictable gradients, which sometimes cause the learning to stop.
2. **Mode Collapse:** The generator, instead of producing diverse data, may only produce a limited number of samples that are acceptable to the discriminator. This problem reduces the diversity of the generated samples.
3. **Vanishing Gradients:** If the discriminator becomes too strong and distinguishes real and generated data well, the gradients provided to the generator become very small or almost zero. This issue leads to the generator's learning stopping.
4. **Saturation of the Cost Function:** In standard GANs, the use of Jensen-Shannon (JS) divergence causes the cost function to saturate and does not provide useful information for improving the model.

How WGAN solves these problems:

1. **Using Wasserstein Distance instead of JS:** The Wasserstein distance is a stronger and differentiable metric for measuring the difference between the distribution of real and generated data. This distance provides meaningful gradients even in conditions where the two distributions do not overlap.
2. **Creating More Stable Gradients:** The Wasserstein distance provides gradients that have smooth and continuous changes and help the generator to continue learning even in complex situations.
3. **More Stability in the Training Process:** In WGAN, a "critic" is used instead of a standard discriminator. The critic is trained to estimate the Wasserstein distance, which leads to a more stable training process.

4.2 Subsection 2

Role of the Critic in WGAN and its Difference with the Discriminator in Standard GANs

In WGAN, the critic, instead of classifying data as real or fake, measures the difference between the distribution of real and generated data. This difference is calculated by estimating the Wasserstein distance. The critic tries to maximize the difference in the scores of real and generated data.

Key Differences between the Role of the Critic in WGAN and the Discriminator in Standard GANs:

1. **Output Type:**
 - In standard GANs, the discriminator provides a probability between 0 and 1, which indicates whether the data is real or fake. This output uses the sigmoid function.
 - In WGAN, the critic provides a real numerical value, which indicates the "score of being real" of the data and lacks a range limitation.
2. **Objective Function:**
 - In standard GANs, the discriminator tries to classify the data into two categories: real and fake. But in WGAN, the critic maximizes the difference between the average scores of real and generated data.

3. Lipschitz Constraint:

- The critic in WGAN must be a 1-Lipschitz function. This condition ensures that the estimation of the Wasserstein distance is done correctly. Usually, this condition is applied through techniques like "weight clipping" or "gradient penalty".
4. **Training Process:** In WGAN, the critic is updated several times relative to the generator to get a more accurate estimate of the Wasserstein distance.

4.3 Subsection 3: Evaluation Metrics

4.3.1 1. Kullback-Leibler Divergence (KL Divergence)

KL Divergence is one of the most famous methods for measuring the difference between two distributions. This metric shows how close a distribution P (real distribution) is to a distribution Q (generated distribution). In simple terms, KL Divergence says that if we want to use distribution Q instead of P, how much information we will lose. The formula for KL Divergence is as follows:

$$D_{KL}(P||Q) = \sum_x P(x) \log \frac{P(x)}{Q(x)}$$

4.3.2 2. Jensen-Shannon Divergence (JS Divergence)

JS Divergence is a modified version of KL Divergence. This metric is designed to solve problems like the asymmetry and sensitivity to zero probability of KL Divergence. In JS Divergence, the average of the two distributions P and Q is used for calculation to solve the asymmetry problem.

4.3.3 3. Wasserstein Distance

The Wasserstein distance or earth mover's distance is specifically designed for measuring the difference between two distributions and is one of the best options for generative models like WGAN. This metric says that to transform a distribution P to a distribution Q (i.e., to move the real data to the generated data), what is the minimum cost we have to pay.

Why is Wasserstein better than KL and JS in WGAN? In WGAN (which is short for Wasserstein GAN), the Wasserstein distance is used to calculate the difference between the real distribution and the generated distribution for the following reasons:

1. **Training Stability:** One of the biggest problems of traditional GANs that use KL or JS Divergence is that their training can be unstable and sometimes quickly run into problems like "mode collapse" (where the model only produces samples from a specific category). The use of the Wasserstein distance reduces these problems and makes the training process more stable.
2. **Improved Convergence:** The Wasserstein distance allows the model to gradually converge to the real distribution. This feature causes the network to slowly learn how to simulate real data and prevents problems like non-convergence or getting lost in training.
3. **Insensitivity to Zero Points:** One of the problems of using KL or JS Divergence is that if the generated distribution Q has a non-zero value at points where the real distribution P has zero probability, these metrics will tend to infinity and disrupt the training. The Wasserstein distance does not have this problem and can effectively manage these differences.

4.4 Subsection 4

4.4.1 1. Loss Value of the Critic

In WGAN, instead of using a typical loss function like in classic GANs (which usually includes a "binary cross-entropy" or "log-likelihood" function), the Wasserstein distance between the real and generated distributions is used. In this way, a "critic" (which is actually a neural network) is trained to evaluate the Wasserstein distance between these two distributions.

- **Wasserstein-1 Distance:** The Wasserstein-1 distance (or Earth Mover’s Distance) between two distributions P_r (real data) and P_g (generated data) is defined as follows:

$$W(P_r, P_g) = \sup_{\|f\|_L \leq 1} E_{x \sim P_r}[f(x)] - E_{x \sim P_g}[f(x)]$$

where f is a function that is constrained to be 1-Lipschitz.

- In WGAN, the goal is for the critic to minimize the loss (or Wasserstein distance). Therefore, if the critic’s loss value is decreasing and eventually approaches a constant value, this indicates that the network is moving towards convergence. For evaluating convergence, the value of this loss is usually monitored over time. A decrease in the loss value means that the generated distribution is getting closer to the real distribution.

4.4.2 2. Sample Density

The density of the data generated by the network is one of the methods for evaluating the quality and convergence of the model. Assume the generator network wants to generate data similar to the real data. If the model is trained correctly, the distribution of the generated data should be similar to the distribution of the real data. To check the sample density, you can use different methods, including:

- **Histograms or Scatter Plots:** You can compare the distribution of the generated data against the distribution of the real data through histogram or scatter plots. If a great similarity is observed between these two distributions, the network has likely approached convergence.

4.4.3 3. Visual Evaluation

In graphic tasks like image generation, one of the best methods for evaluating convergence is the visual observation of the generated samples. If the network is trained well, the generated images should have a great similarity to the real data and should generally be identifiable and meaningful. Especially in image generator networks, if the network effectively converges to the real data distribution, similar features should be observed in the generated images. For example:

- In a generative image model, if the generated images find clear similarities to the real data (like objects, colors, textures, etc.), this indicates progress in convergence.
- **Visual Turing Test:** A visual evaluation metric similar to the visual Turing test, in which humans can distinguish the generated images from the real images or not. If humans cannot distinguish the difference between the real and generated images, this is a sign of good convergence.

4.4.4 4. Using Additional Metrics like FID (Fréchet Inception Distance)

FID is one of the famous metrics in evaluating the quality of generated images, which is used by Inception v3 to extract features from images. This metric calculates a distance between the feature distribution of real images and generated images. A decrease in the FID value over time means that the generated images are getting closer to the features of the real data. A low FID value (e.g., below 10) indicates that the model has converged well and the generated images have a similarity to the real images.

4.4.5 5. Monitoring Convergence in Critic and Generator

In WGAN, the critic and generator are trained simultaneously, but these two networks may converge at different speeds. In some cases, the critic may converge faster than the generator, and in this case, it is recommended that the number of critic updates be more than the generator’s.

- **Overtraining of Critic:** If the critic is trained too much (i.e., the number of updates for the critic is much more than the generator), the generator’s performance may become weak and it may not converge. In such conditions, a suitable balance must be established between the number of updates for the critic and the generator.
- **Monitoring the Convergence of Critic and Generator:** Checking the amount of change in the weights and the loss function of each of these two models helps you to realize whether both networks are converging or not.

5 Question 2 - Implementation of GAN Models

5.1 Subsection 1

In this part, we designed the GAN model with this structure:

```
1 transform = transforms.Compose([
2     transforms.Resize(64),
3     transforms.ToTensor(),
4     transforms.Normalize([0.5], [0.5])
5 ])
```

1. **Resize:** Changes the size of the input images to a fixed size of 64x64 so that all images are the same size and compatible with the model.
2. **ToTensor:** Converts the images, which are in the form of raw data (like pixel arrays), into a format processable by PyTorch.
3. **Normalize:** Changes the pixel values from the range [0, 255] to the range [-1, 1] so that the model's learning is faster and more stable.

```
1 class Generator(nn.Module):
2     # ... Generator architecture ...
3     def forward(self, input):
4         return self.model(input)
5
6 class Discriminator(nn.Module):
7     # ... Discriminator architecture ...
8     def forward(self, input):
9         return self.model(input).view(-1, 1)
```

This is the main class:

```
1 class GAN:
2     def __init__(self, latent_dim=100, lr=0.0002):
3         self.generator = Generator(latent_dim).to(device)
4         self.discriminator = Discriminator().to(device)
5         self.latent_dim = latent_dim
6         self.criterion = nn.BCELoss()
7         self.optimizer_G = torch.optim.Adam(self.generator.parameters(), lr=lr, betas
8         =(0.5, 0.999))
9         self.optimizer_D = torch.optim.Adam(self.discriminator.parameters(), lr=lr,
10        betas=(0.5, 0.999))
```

In this part, we trained the model.

```
1 for epoch in range(epochs):
2     for i, (imgs, _) in enumerate(dataloader):
3         # ... Training loop for Generator and Discriminator ...
4
5         # Print progress
6         if i % 100 == 0:
7             print(f"[Epoch {epoch+1}/{epochs}] [Batch {i}/{len(dataloader)}] "
8                   f"[D loss: {d_loss.item():.4f}] [G loss: {g_loss.item():.4f}]")
9
10        # Generate and save images
11        with torch.no_grad():
12            gen_imgs = gan.generator(fixed_noise).cpu()
13            # ... code to save and show images ...
```

5.1.1 GAN Network Implementation on Fashion MNIST Data

1. Generator (Generator)

- **Task:** Generating new images from a random noise vector.
- **Structure:** Consists of several `ConvTranspose2d` layers that gradually magnify the input noise vector and convert it into an image with dimensions 64x64.
- **Activation Functions:**

- **ReLU** is used in the middle layers for activation.
- **Tanh** is used in the output layer to normalize the pixel values of the generated image to the range $[-1, 1]$.

2. Discriminator (Discriminator)

- **Task:** Identifying real images from images generated by the generator.
- **Structure:** Consists of several **Conv2d** layers that gradually reduce the size of the input image and extract its important features.
- **Activation Functions:**
 - **LeakyReLU** is used to prevent the problem of vanishing gradients. This function, compared to **ReLU**, allows a small amount of gradient to pass in the negative part.
 - **Sigmoid** is used in the output layer to calculate the probability of the image being real between 0 and 1.

3. GAN (Model Management)

- **Task:** Managing the generator and discriminator networks, setting the cost functions, and optimization.
- **Details:**
 - **Binary Cross-Entropy Loss** is used to calculate the errors.
 - * **Generator:** Tries to reduce the discriminator's error so that the generated images look more real.
 - * **Discriminator:** Tries to distinguish real and fake images with more accuracy.
 - The Adam optimizer has been used, which provides stability and high speed in training due to its specific settings (like adaptive learning rate).

5.2 Subsection 2

5.2.1 Explaining the ConvTranspose2d Layer in the Generator

1. Role of the ConvTranspose2d Layer in Neural Networks

- **Main Application:** The **ConvTranspose2d** layer, sometimes known as "Deconvolution" or "Transposed Convolution", is used for upsampling images or data.
- **Difference with Conv2d:** While **Conv2d** is used to reduce dimensions and extract features, **ConvTranspose2d** acts in reverse and increases the dimensions of the input data.
- **Role in GANs:** In generator networks, this layer is used to transform a noise vector (a small random vector) into larger images with desired dimensions, like 64x64 pixels.

2. Changing the Dimensions of Input Data by ConvTranspose2d

- **How it works:** This layer uses a convolutional kernel and increases the image dimensions by expanding each position in the input.
- **Example:** An input with dimensions 1x1 (a noise vector) is sequentially transformed to 4x4, then 8x8, and finally to 64x64.

3. Effect of ConvTranspose2d Parameters on the Output

Several important parameters of this layer are:

1. **Kernel Size:** Determines the size of the region used for convolution in each step.
2. **Stride:** Determines how much the input layer is shifted in each step. A larger stride value leads to a faster increase in the output dimensions.
3. **Padding:** Adds zeros around the input data to control the output dimensions.
4. **Output Padding:** For precise adjustment of the output dimensions if needed.

4. Formula for Calculating Output Dimensions The output dimensions H_{out} and W_{out} for the `ConvTranspose2d` layer are calculated as follows:

$$H_{out} = (H_{in} - 1) \times \text{stride} - 2 \times \text{padding} + \text{kernel_size} + \text{output_padding}$$

$$W_{out} = (W_{in} - 1) \times \text{stride} - 2 \times \text{padding} + \text{kernel_size} + \text{output_padding}$$

Parameters:

- H_{in}, W_{in} : Input height and width.
- `stride`: Convolution step.
- `padding`: Padding added to the input.
- `kernel_size`: Kernel size.
- `output_padding`: Final padding to adjust the output dimensions.

5.3 Subsection 3

```
[Epoch 1/10] [Batch 0/938] [D loss: 0.6795] [G loss: 0.6267]
[Epoch 1/10] [Batch 100/938] [D loss: 0.0013] [G loss: 6.2292]
[Epoch 1/10] [Batch 200/938] [D loss: 0.0377] [G loss: 13.4333]
[Epoch 1/10] [Batch 300/938] [D loss: 0.0849] [G loss: 2.5734]
[Epoch 1/10] [Batch 400/938] [D loss: 0.3171] [G loss: 1.1969]
[Epoch 1/10] [Batch 500/938] [D loss: 0.6135] [G loss: 0.4932]
[Epoch 1/10] [Batch 600/938] [D loss: 0.2599] [G loss: 1.8977]
[Epoch 1/10] [Batch 700/938] [D loss: 0.3166] [G loss: 2.0946]
[Epoch 1/10] [Batch 800/938] [D loss: 0.3741] [G loss: 1.2295]
[Epoch 1/10] [Batch 900/938] [D loss: 0.4286] [G loss: 3.1867]
```

Epoch 1

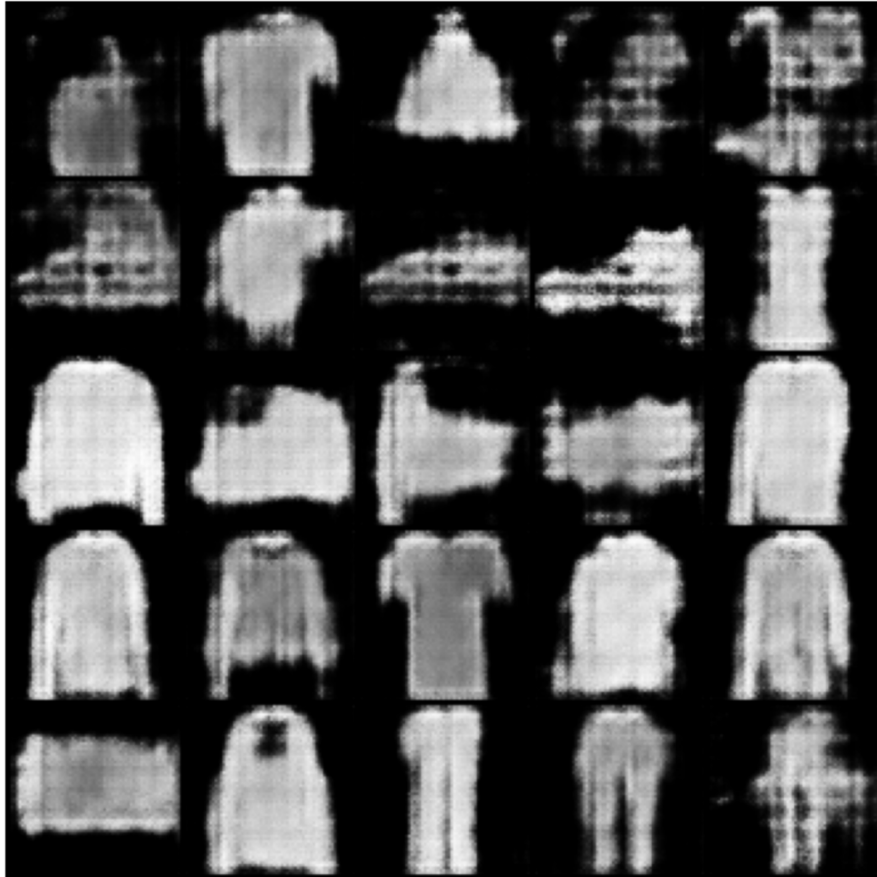


Figure 4: Epoch 1

```

[Epoch 5/10] [Batch 0/938] [D loss: 0.0929] [G loss: 2.7903]
[Epoch 5/10] [Batch 100/938] [D loss: 0.0644] [G loss: 5.1395]
[Epoch 5/10] [Batch 200/938] [D loss: 0.0356] [G loss: 9.1352]
[Epoch 5/10] [Batch 300/938] [D loss: 0.0611] [G loss: 3.5596]
[Epoch 5/10] [Batch 400/938] [D loss: 0.2820] [G loss: 5.9128]
[Epoch 5/10] [Batch 500/938] [D loss: 0.0286] [G loss: 3.4545]
[Epoch 5/10] [Batch 600/938] [D loss: 0.0779] [G loss: 3.1224]
[Epoch 5/10] [Batch 700/938] [D loss: 0.0149] [G loss: 5.2746]
[Epoch 5/10] [Batch 800/938] [D loss: 0.0673] [G loss: 4.9205]
[Epoch 5/10] [Batch 900/938] [D loss: 0.0418] [G loss: 4.1381]

```

Epoch 5



Figure 5: Epoch 5

```

[Epoch 10/10] [Batch 0/938] [D loss: 0.1201] [G loss: 2.8691]
[Epoch 10/10] [Batch 100/938] [D loss: 0.2373] [G loss: 1.9228]
[Epoch 10/10] [Batch 200/938] [D loss: 0.2901] [G loss: 1.7430]
[Epoch 10/10] [Batch 300/938] [D loss: 0.0168] [G loss: 4.3346]
[Epoch 10/10] [Batch 400/938] [D loss: 0.0110] [G loss: 4.9771]
[Epoch 10/10] [Batch 500/938] [D loss: 0.0115] [G loss: 6.4616]
[Epoch 10/10] [Batch 600/938] [D loss: 0.1117] [G loss: 4.1411]
[Epoch 10/10] [Batch 700/938] [D loss: 0.0411] [G loss: 3.6966]
[Epoch 10/10] [Batch 800/938] [D loss: 0.0461] [G loss: 3.6132]
[Epoch 10/10] [Batch 900/938] [D loss: 0.0333] [G loss: 4.4033]

```

Epoch 10



Figure 6: Epoch 10

5.3.1 2. Generating and Displaying Samples

To check the output quality during training, we use a fixed noise vector (`fixed_noise`). This allows for a direct comparison of the Generator's outputs in different epochs.

5.4 Subsection 4: Plotting Generator and Discriminator Loss

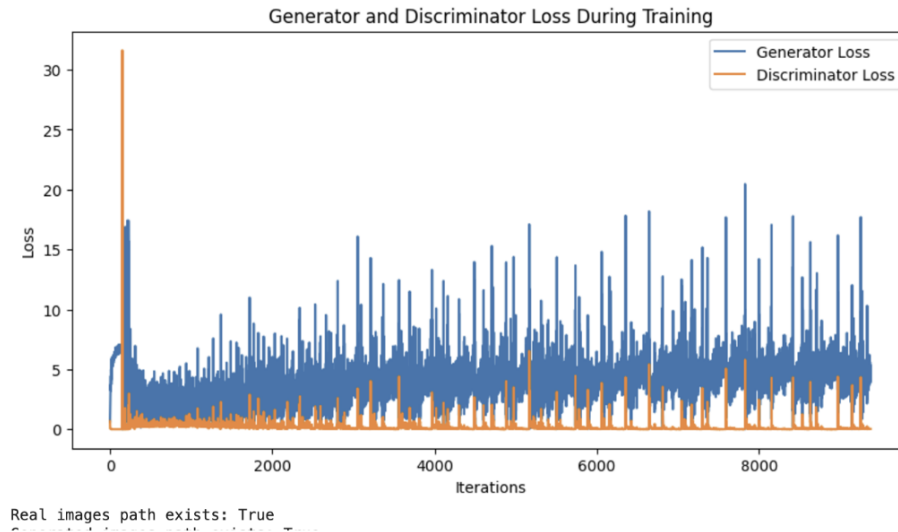


Figure 7: Generator and Discriminator Loss During Training

Analysis of the Charts: In the chart, we see that initially, in the early iterations that occurred in the first epoch, the generator loss was decreasing, which means better images were being produced by the generator. And the discriminator also had a high loss initially and was decreasing at each stage, which indicates that the discriminator was learning to distinguish fake and real data. Further on, we see that both are competing with each other, and in the end, the discriminator has a lower loss, and the generator had a higher loss, which indicates that because we don't have good data and structure, the generator has not produced good data, although the generated images were recognizable by humans. In general, the further we went, the stronger the discriminator became, and the generator also produced better images, but the discriminator became stronger than it and overcame it.

5.5 Subsection 5: FID Explanations

5.5.1 FID (Fréchet Inception Distance) Metric

Fréchet Inception Distance (FID) is one of the most reputable metrics for evaluating the quality and diversity of images generated by generative networks like GANs. This metric is specifically designed to check the similarity between the distribution of real images and generated images and uses features extracted by a pre-trained model like the Inception Network.

1. Use of FID in Evaluating the Quality of Generated Images Main Application of FID:

- **Measuring the Quality of Generated Images:** FID measures the similarity of the feature distribution of real and generated images. These features are extracted from the middle layers of a deep network (usually the Inception network), which contain high-level information like structure, shape, and texture of the image.
- **Detecting Image Diversity:** In addition to quality, FID evaluates the coverage of the real data distribution. In other words, if the Generator only produces a limited number of samples, FID detects this problem.

Advantages of FID over other metrics:

1. **Detecting Details:** FID, due to the use of high-level features extracted from images, can detect subtle differences in the quality and structure of images.
2. **Flexibility:** This metric can be used for different types of datasets and is not sensitive to changes in scale or rotation of images.
3. **Less Sensitivity to Repetitive or Copied States:** If the Generator produces repetitive images (like the "mode collapse" problem), FID clearly shows this defect.

2. FID Mathematical Formula and How it Works FID Formula:

$$FID(P_r, P_g) = \|\mu_r - \mu_g\|^2 + \text{Tr}(\Sigma_r + \Sigma_g - 2(\Sigma_r \Sigma_g)^{1/2})$$

Formula Components:

1. μ_r, μ_g : The mean vectors of the features extracted from the real (P_r) and generated (P_g) images. This part shows the difference in the average features of the real and generated images.
2. Σ_r, Σ_g : The covariance matrices of the features of the real and generated images. This part shows the difference in the diversity and correlation of the features in the two distributions.
3. Tr : Represents the trace of the matrix (sum of the diagonal elements).

Mathematical Explanation of the Formula:

1. **Difference of Means ($\|\mu_r - \mu_g\|^2$):** The first part of the formula measures the Euclidean distance between the average features of the real and generated images. The smaller this value, the closer the average features in the two distributions are to each other.
2. **Difference of Covariances ($\text{Tr}(\dots)$):** This part is more complex and examines the difference in the structure and diversity of the real and generated data.
 - The calculation of the matrix square root $(\Sigma_r \Sigma_g)^{1/2}$ represents the overlap between the two covariance distributions.
 - The smaller this value, the more similar the covariance matrices of the real and generated data are.

3. How FID Works in Practice Steps to Calculate FID:

1. **Feature Extraction:** Real and generated images are passed through a pre-trained network (like Inception v3). The features are usually extracted from one of the final layers (e.g., before the Softmax layer).
2. **Calculating the Mean Vector (μ) and Covariance Matrix (Σ):** For each set (real and generated images), the mean vector and covariance matrix of the features are calculated.
3. **Applying the FID Formula:** Using the calculated values, the final FID value is determined.

4. Analysis of the FID Concept

- **Lower FID:** A lower FID value indicates a greater similarity between the feature distribution of the real and generated images. In other words, the Generator has been able to produce more realistic images.
- **Higher FID:** A higher FID value means a greater difference in the quality and diversity of the generated images compared to the real images. This can be due to problems like "mode collapse" or the generation of low-quality images.

6. Application of FID in GANs

1. Evaluating Quality in Different Training Stages:

- In the initial stages, FID is usually high because the Generator produces low-quality images.
 - With the progress of training, the FID value decreases, which indicates an improvement in the quality of the generated images.
2. **Comparing Models:** FID is used as a standard metric for comparing the quality of images generated in different GAN architectures.

5.6 Subsection 6

In this part, we will calculate the FID corresponding to each epoch:

```
1 # Calculate FID score
2 fid_value = fid_score.calculate_fid_given_paths([real_path, generated_epoch_dir],
3         batch_size, device, dims=2048)
4 FID_scores.append(fid_value)
5 print(f"FID Score at Epoch {epoch+1}: {fid_value}")
```

Before that, a number of images must be created for FID.

```
1 generated_epoch_dir = os.path.join(generated_path_epoch, f'epoch_{epoch+1}')
2 os.makedirs(generated_epoch_dir, exist_ok=True)
3 with torch.no_grad():
4     for j in range(0, len(dataloader.dataset), batch_size):
5         noise = torch.randn(batch_size, latent_dim, 1, 1, device=device)
6         gen_imgs = gan.generator(noise).cpu()
7         for k, img in enumerate(gen_imgs):
8             torchvision.utils.save_image(img, os.path.join(generated_epoch_dir, f'img_{j
9                 +k}.png'), normalize=True)
```

And finally, we display it.

```
1 plt.figure(figsize=(10, 5))
2 plt.title("FID Score During Training")
3 plt.plot(range(1, epochs+1), FID_scores, marker='o', label="FID Score")
4 plt.xlabel("Epoch")
5 plt.ylabel("FID Score")
6 plt.legend()
7 plt.grid()
8 plt.show()
```

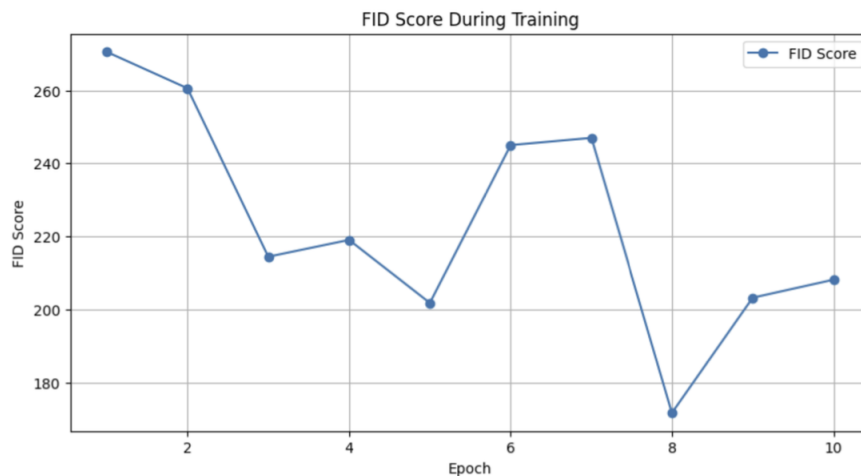


Figure 8: FID Score During Training

The FID chart fluctuates, which is probably due to the MinMax nature of GAN models. We calculated the FID on 16 images from each of the main and fake distributions each time, which is why it probably couldn't estimate the exact FID value. If we had applied FID on a larger number of data, we would have reached a better result, which would also take a much longer time. Because it takes a very long time, we only compared 16 data in each epoch.

References

- [1] Goodfellow, I., et al. (2014). Generative Adversarial Nets. *Advances in Neural Information Processing Systems, 27*, 2672-2680.
- [2] Salimans, T., et al. (2016). Improved Techniques for Training GANs. *Advances in Neural Information Processing Systems, 29*, 2234-2242.
- [3] Arjovsky, M., Chintala, S., & Bottou, L. (2017). Wasserstein GAN. *arXiv preprint arXiv:1701.07875*.
- [4] Behrmann, J., Grathwohl, W., Chen, R. T. Q., Duvenaud, D., Jacobsen, J.-H. (2019). Invertible Residual Networks. *International Conference on Machine Learning (ICML)*.
- [5] Chen, R. T. Q., Behrmann, J., Duvenaud, D., Jacobsen, J.-H. (2019). Residual Flows for Invertible Generative Modeling. *Advances in Neural Information Processing Systems (NeurIPS)*.
- [6] Hutchinson, M. F. (1990). A Stochastic Estimator of the Trace of the Influence Matrix for Laplacian Smoothing Splines. *Communications in Statistics – Simulation and Computation*.